Experiment 1: Average Score of Each Subject and Highest Average

Aim:

To calculate the average marks of each subject using NumPy arrays and identify the subject with the highest average marks.

Algorithm:

1. Import the NumPy library.
2. Create a 4×4 NumPy array containing marks of four students in Math, Science, English, and History.
3. Calculate the average marks for each subject using np.mean() with axis=0.
4. Find the highest average using np.max().
5. Identify the subject with the highest average using np.argmax().
6. Display the averages and the subject with the highest average.

Python Code (IDLE):

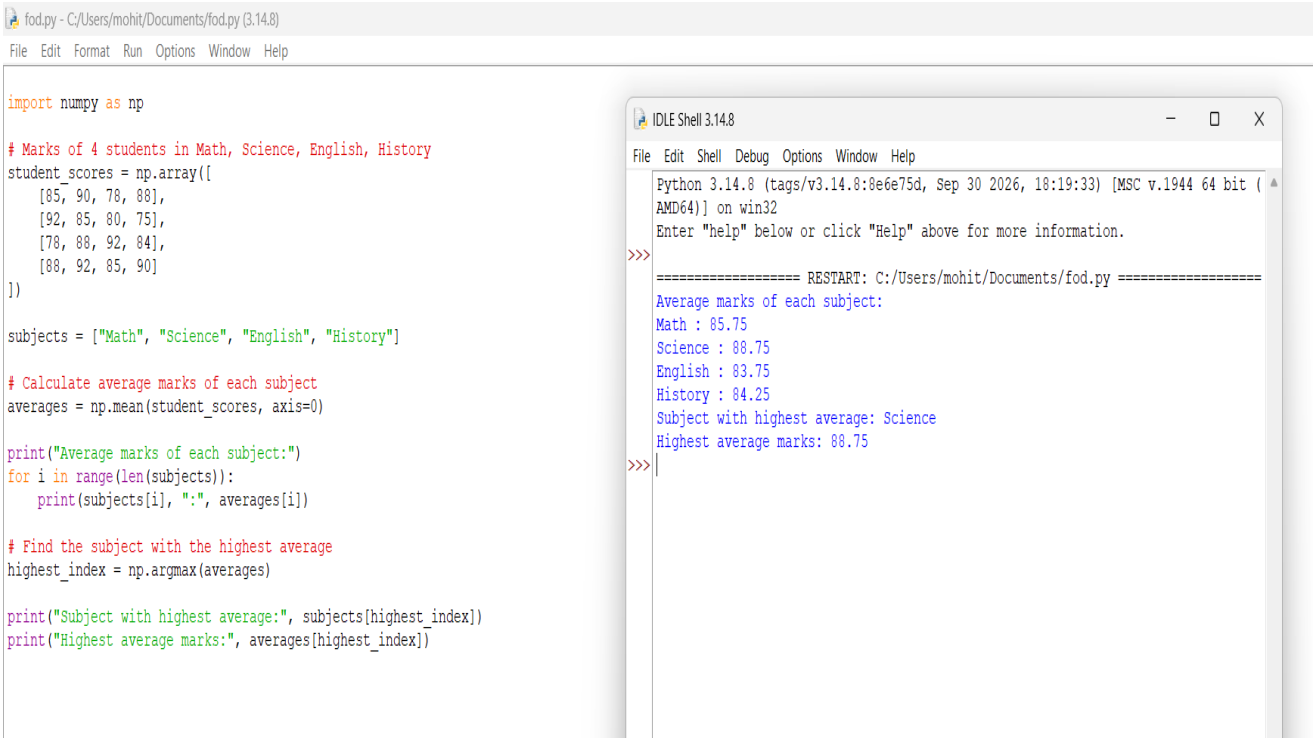
```
import numpy as np
# Marks of 4 students in Math, Science, English, Historystudent_scores =
np.array([
    [85, 90, 78, 88],
    [92, 85, 80, 75],
    [78, 88, 92, 84],
    [88, 92, 85, 90]
])
subjects = ["Math", "Science", "English", "History"]
# Calculate average marks of each subjectaverages =
np.mean(student_scores, axis=0)
print("Average marks of each subject:")for i in range(len(subjects)):
    print(subjects[i], ":", averages[i])
# Find the subject with the highest averagehighest_index =
np.argmax(averages)
print("Subject with highest average:",
subjects[highest_index])print("Highest average marks:",
averages[highest_index])
```

Input:

```
[[85, 90, 78, 88],
 [92, 85, 80, 75],
 [78, 88, 92, 84],
```

[88, 92, 85, 90]]

Output:



Result:

Thus, the average marks of each subject were calculated using NumPy, and Science was identified as the subject with the highest average of 88.75 marks.

Experiment 2: Average Price of Products Sold

Aim:

To calculate the average price of all products sold during the past month using NumPy arrays.

Algorithm:

1. Import the NumPy library.

2. Create a 3×3 NumPy array representing sales data for three products.

3. Store the product prices in a separate NumPy array.

4. Calculate the average price using np.mean().

5. Display the average product price.

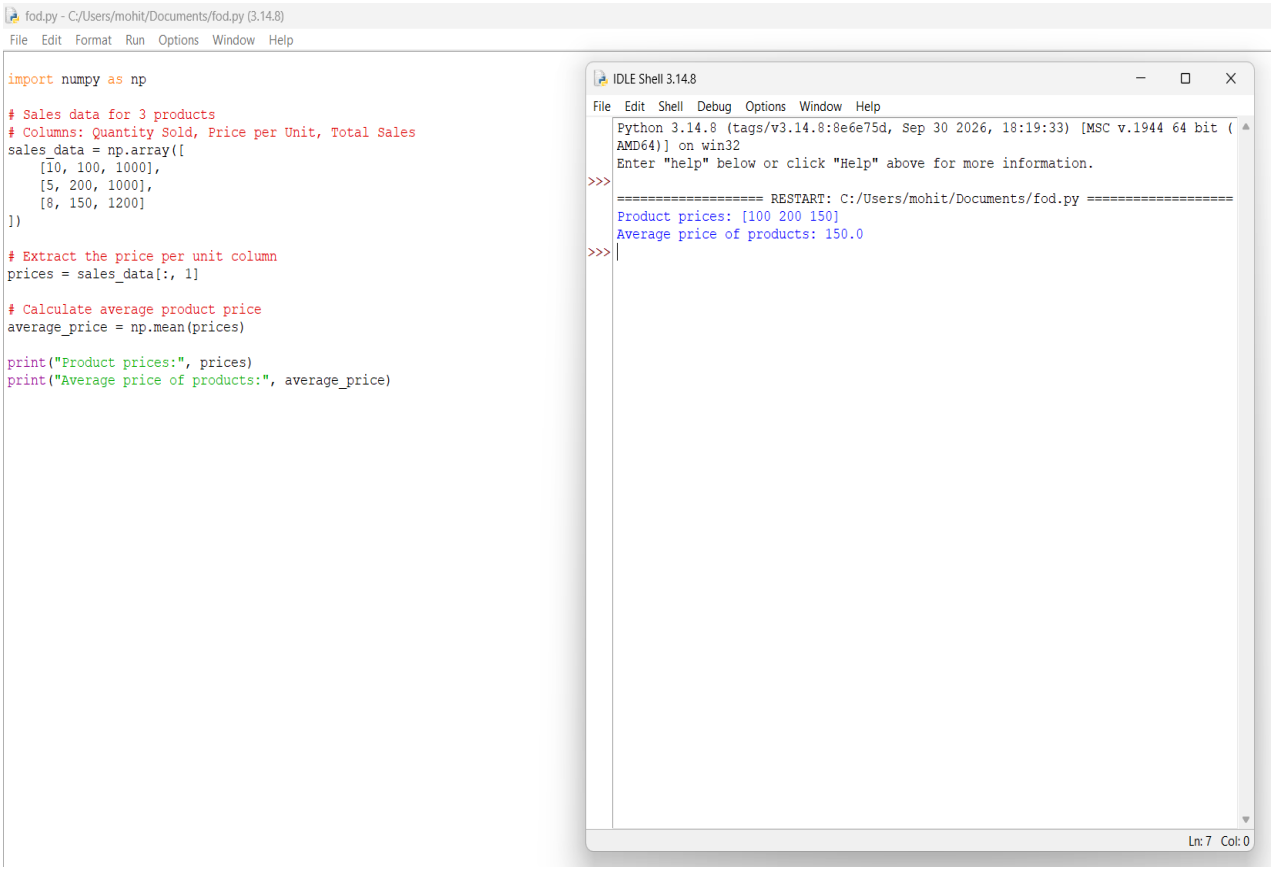
Python Code (IDLE):

```
import numpy as np
# Sales data for 3 products# Columns: Quantity Sold, Price per Unit, Total Salessales_data = np.array([
    [10, 100, 1000],
    [5, 200, 1000],
    [8, 150, 1200]
])
# Extract the price per unit columnprices = sales_data[:, 1]
# Calculate average product priceaverage_price = np.mean(prices)
print("Product prices:", prices)print("Average price of products:", average_price)
```

Input:

```
[[10, 100, 1000],
 [5, 200, 1000],
 [8, 150, 1200]]
```

Output:



Result:

Thus, the average price of the three products was calculated using NumPy and found to be 150.0.

Experiment 3: Average Sale Price of Houses with More Than Four Bedrooms

Aim:

To find the average sale price of houses having more than four bedrooms using NumPy arrays and operations.

Algorithm:

1. Import the NumPy library.

2. Create a NumPy array containing the number of bedrooms, square footage, and sale price of houses.
3. Select houses with more than four bedrooms using a Boolean condition.
4. Extract the sale prices of the selected houses.
5. Calculate the average sale price using np.mean().
6. Display the average sale price.

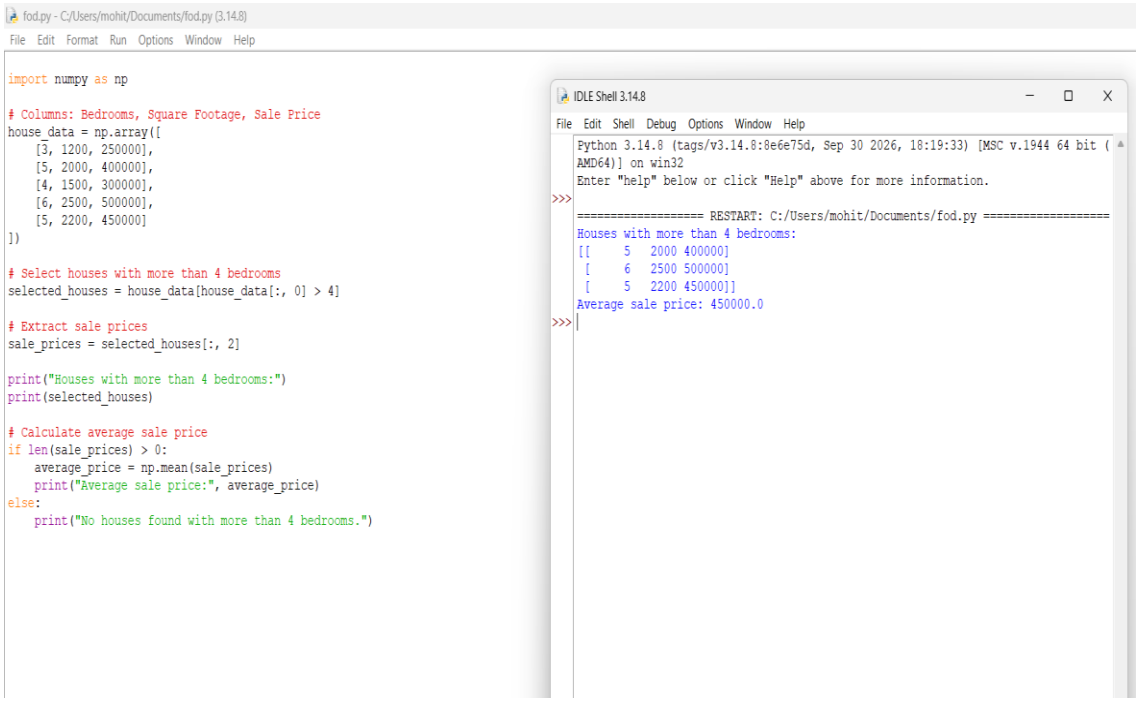
Python Code (IDLE):

```
import numpy as np
# Columns: Bedrooms, Square Footage, Sale Pricehouse_data =
np.array([
    [3, 1200, 250000],
    [5, 2000, 400000],
    [4, 1500, 300000],
    [6, 2500, 500000],
    [5, 2200, 450000]
])
# Select houses with more than 4 bedroomsselected_houses =
house_data[house_data[:, 0] > 4]
# Extract sale pricessale_prices = selected_houses[:, 2]
print("Houses with more than 4 bedrooms:")print(selected_houses)
# Calculate average sale priceif len(sale_prices) > 0:
    average_price = np.mean(sale_prices)
    print("Average sale price:", average_price)else:
    print("No houses found with more than 4 bedrooms.")
```

Input:

[[3, 1200, 250000],
 [5, 2000, 400000],
 [4, 1500, 300000],
 [6, 2500, 500000],
 [5, 2200, 450000]]

Output:



Result:

Thus, houses with more than four bedrooms were selected using NumPy Boolean indexing, and their average sale price was calculated as 450,000.